

I metodi iterativi

In questo capitolo ci occuperemo di un altro modo per risolvere sistemi lineari: i metodi iterativi. Come abbiamo fatto nel capitolo precedente daremo per ognuno dei metodi analizzati, una descrizione dettagliata ed esaustiva in modo tale da rendere l'implementazione più agevole nel linguaggio di programmazione scelto.

Consideriamo un sistema lineare

$$A\mathbf{x} = \mathbf{b},$$

con $A \in \mathbb{R}^{n \times n}$ non singolare, $\mathbf{b} \in \mathbb{R}^n$ è termine noto e $\mathbf{x} \in \mathbb{R}^n$ è il vettore delle incognite. I metodi iterativi partendo da un vettore iniziale $\mathbf{x}^{(0)} \in \mathbb{R}^n$ creano una sequenza di vettori $\mathbf{x}^{(k)} \in \mathbb{R}^n$ che si avvicinano sempre di più alla soluzione esatta $\tilde{\mathbf{x}} \in \mathbb{R}^n$. L'idea alla base è chiara si però aprono i seguenti quesiti:

- ◊ Perché introdurre questo tipo di metodi?
- ◊ Come è possibile stabilire se si è arrivati vicini alla soluzione dato che non si conosce la soluzione esatta $\tilde{\mathbf{x}}$?
- ◊ È sempre possibile arrivare alla soluzione indipendentemente dal punto di partenza?
- ◊ Come si può creare una tale successione di vettori?

4.1. Fill-in

Nella maggior parte delle applicazioni la matrice A ha dimensione molto elevata dell'ordine di 10^6 . Abbiamo già visto che salvare l'intera matrice con un formato standard "Column-major" o "Row-major" richiederebbe l'utilizzo di molta memoria.

Se considerassimo per esempio una matrice quadrata di dimensione 10^6 e se per ogni sua entrata ci servissero 8 bit, avremmo bisogno di $8e + 10^{12}$ bit e software come Matlab[®] non permettono nemmeno di creare matrici così grandi.

Dato che nella maggior parte delle applicazioni matrici di così grande dimensione sono sparse, si può ovviare al problema del salvataggio tramite la rappresentazione sparsa delle matrici.

Consideriamo per esempio la matrice $A \in \mathbb{R}^{100 \times 100}$, le cui entrate non nulle sono rappresentate in Figura 1.

Quando viene fatta la decomposizione PALU o quella di Cholesky è possibile che le matrici risultanti perdano la loro natura sparsa. Questo fenomeno è noto come fill-in. In Figura 2 riportiamo lo schema della sparsità delle matrici $L, U \in \mathbb{R}^{100 \times 100}$, mentre in Figura 3 riportiamo quello associato alla matrice $R \in \mathbb{R}^{100 \times 100}$ della decomposizione di Cholesky. Il numero di

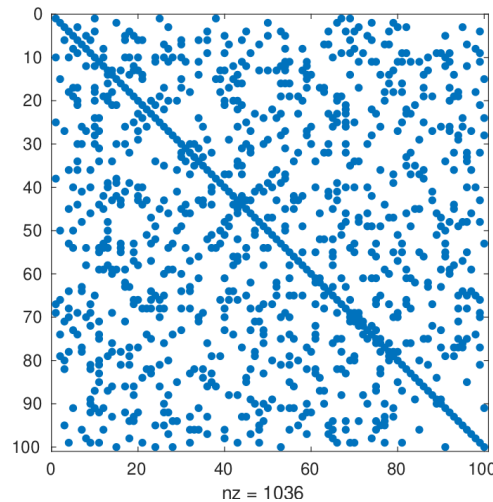


Figura 1. Rappresentazione tramite il comando `spy` di Matlab[®] della matrice sparsa A presa in esame. Vengono anche riportate le sue entrate non nulle 1036 (circa il 10% delle entrate totali della matrice).

entrate non nulle aumenta notevolmente, diventa più del triplo, quindi possiamo dire che la matrice A è soggetta al fenomeno di fill-in per entrambe le decomposizioni.

Di conseguenza anche qualora fossimo in grado di creare queste decomposizioni, esse potrebbero aumentare le entrate non nulle delle matrici. Tale incremento potrebbe far perdere l'efficacia della rappresentazione sparsa della matrice e quindi bloccare l'intera procedura solamente per questioni di memoria.

Vedremmo che le procedure iterative tendono a preservare la natura sparsa delle matrici. Infatti esse sfruttano le operazioni base di somma e prodotto fra matrici senza alcuna decomposizione e quindi senza incombere nel rischio del fill-in.

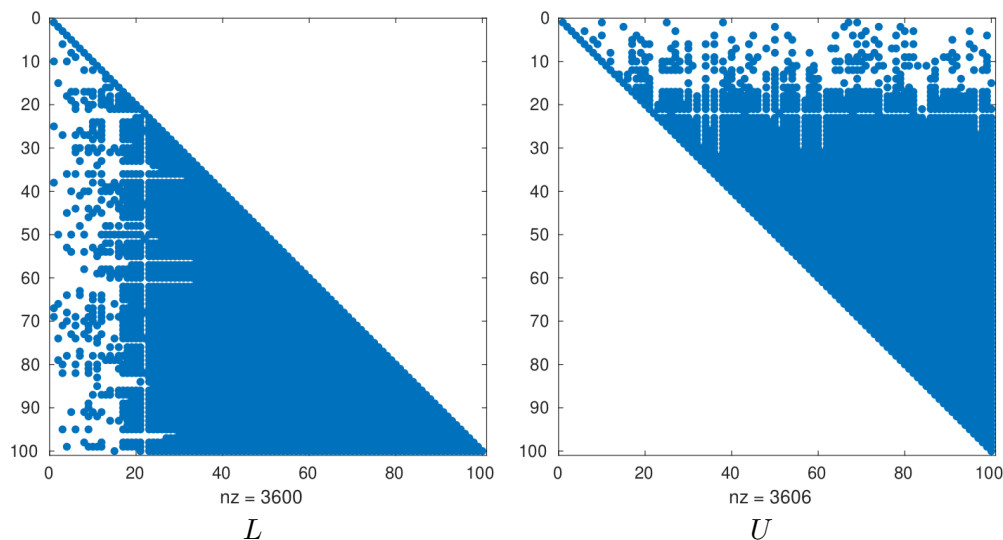


Figura 2. Rappresentazione tramite il comando `spy` di Matlab[®] delle matrici L e U risultanti dalla decomposizione **PALU**. Vengono anche riportate le sue entrate non nulle di L 3600, e quelle di U 3606 (circa il 36% delle entrate totali della matrice sia per L che per U).

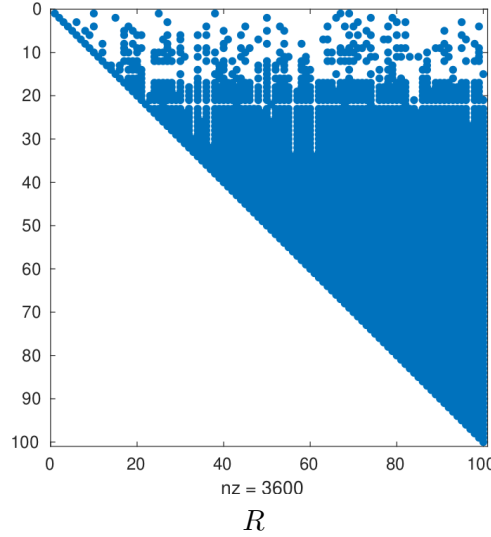


Figura 3. Rappresentazione tramite il comando `spy` di Matlab[®] della matrice R risultanti dalla decomposizione Cholesky. Vengono anche riportate le sue entrate non nulle 3600 (circa il 36% delle entrate totali della matrice).

4.2. Criterio di arresto

I metodi di risoluzione di sistemi lineari presentati in questo capitolo sono iterativi ossia, partendo da un primo tentativo di soluzione del sistema lineare, $\mathbf{x}^{(0)} \in \mathbb{R}^n$, creano una sequenza di vettori $\mathbf{x}^{(k)} \in \mathbb{R}^n$ che si avvicinano sempre di più alla soluzione esatta.

In generale tutti gli algoritmi iterativi hanno una quantità molto importante chiamata tolleranza, `tol`, che identifica quando la soluzione approssimata $\mathbf{x}^{(k)}$ è vicina alla soluzione esatta.

Per misurare questa vicinanza si sfrutta la norma di vettori. Purtroppo non è possibile utilizzare la soluzione esatta $\tilde{\mathbf{x}} \in \mathbb{R}^n$ e bloccare le iterazioni quando

$$\frac{\|\mathbf{x}^{(k)} - \tilde{\mathbf{x}}\|}{\|\tilde{\mathbf{x}}\|} < \text{tol}.$$

Per trovare la soluzione di un sistema lineare tramite una procedura iterativa, si considerano due criteri indipendentemente dal metodo utilizzato:

(1) **incremento tra due iterate:** a ogni iterazione viene calcolato

$$(4.1) \quad \frac{\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|}{\|\mathbf{x}^{(0)}\|},$$

dove $\mathbf{x}^{(k)}$ è la soluzione calcolata al passo k e $\mathbf{x}^{(k-1)}$ è la soluzione calcolata al passo precedente. Possiamo vedere la quantità di Equazione (4.1) come quanto due iterate successive differiscono fra loro. Se questa quantità è piccola, o per meglio dire più piccola di una tolleranza fissata, i due vettori $\mathbf{x}^{(k)}$ e $\mathbf{x}^{(k-1)}$ si possono considerare “uguali”.

(2) **residuo scalato:** a ogni iterazione viene calcolato

$$(4.2) \quad \frac{\|\mathbf{b} - A\mathbf{x}^{(k)}\|}{\|\mathbf{b}\|},$$

dove A e $\mathbf{b}$ sono esattamente la matrice e il termine noto associati al problema che stiamo risolvendo. Supponiamo che $\tilde{\mathbf{x}}$ sia la soluzione esatta e riscriviamo il sistema

$$A\tilde{\mathbf{x}} = \mathbf{b} \quad \Rightarrow \quad \mathbf{b} - A\tilde{\mathbf{x}} = \mathbf{0}.$$

Prendendo in esame solamente il numeratore di Equazione (4.2) e togliendo la norma otteniamo qualcosa di simile

$$\mathbf{b} - A\mathbf{x}^{(k)}.$$

Di conseguenza richiedere che la quantità in Equazione (4.2) è minore della tolleranza `tol`, vuol dire che il prodotto

$$A\mathbf{x}^{(k)} \approx \mathbf{b},$$

ossia $\mathbf{x}^{(k)}$ risolve il sistema lineare per con la precisione `tol` che abbiamo richiesto. In Equazione (4.2) abbiamo diviso per la norma di $\mathbf{b}$ per adimensionalizzare questa quantità e ottenere una sorta di “errore relativo”, confronta questa equazione con Equazione (2.2) nella definizione di errore relativo. Un altro modo è quello di normalizzare rispetto al residuo ottenuto al primo passo.

Osservazione 4.1. *Proprio come il calcolo dell'errore relativo anche in questo caso i criteri d'arresto possono utilizzare una qualsiasi delle norme introdotte nella Sezione 1.2.*

Rapidità dei criterio di arresto. Consideriamo il criterio di arresto basato sull'incremento tra due iterate successive. Non abbiamo guardato nello specifico, ma il calcolo di una somma/differenza di vettori di dimensioni costa n somme, un'operazione per ogni entrata, quindi $O(n)$. Poi, il calcolo della norma richiede $O(n)$ operazioni. Di conseguenza il costo complessivo è

$$\underbrace{O(n)}_{\text{somma}} + 2 \underbrace{O(n)}_{\text{norma}} + \underbrace{1}_{\text{divisione}} = O(n).$$

Il criterio basato sul residuo è più oneroso e complesso. Infatti esso coinvolge

- (1) il prodotto fra la matrice A e il vettore $\mathbf{x}^{(k)}$, ossia $O(n^2)$;
- (2) la somma fra il vettore $A\mathbf{x}^{(k)}$ e $\mathbf{b}$, $O(n)$;
- (3) il calcolo di due norme $\|\mathbf{b} - A\mathbf{x}^{(k)}\|$ e $\|\mathbf{b}\|$, nel caso della norma 2, sappiamo costare $O(n)$;
- (4) un rapporto, $\|\mathbf{b} - A\mathbf{x}^{(k)}\|/\|\mathbf{b}\|$.

Riassumendo il costo totale è

$$\underbrace{O(n^2)}_{(1)} + \underbrace{O(n)}_{(2)} + 2 \underbrace{O(n)}_{(3)} + \underbrace{1}_{(4)} = O(n^2).$$

Il costo è più alto rispetto a quello dell'incremento su due iterate successive, $O(n)$ contro $O(n^2)$. Tuttavia rimane comunque polinomiale e con un grado basso. Inoltre vedremo che alcuni metodi iterativi calcoleranno già la quantità $\mathbf{b} - A\mathbf{x}^{(k)}$ per ottenere l'iterata successiva $\mathbf{x}^{(k+1)}$. Di conseguenza la parte più onerosa per il calcolo del residuo è inglobato nel calcolo delle iterate.

Nota Implementativa 4.1. *Quando si calcola il residuo si calcola sempre la norma di $\mathbf{b}$. Il termine noto non varia durante tutte le iterazioni, quindi può essere calcolato solamente la prima volta.*

Nota Implementativa 4.2. *Nella descrizione della rapidità del calcolo dei criteri d'arresto non abbiamo tenuto conto del fatto che il prodotto matrice-vettore è ottimizzato visto che la matrice $\mathbf{A}$ ha una struttura sparsa. Di conseguenza il prodotto matrice-vettore non ha in realtà un costo $O(n^2)$, ma è proporzionale al numero di entrate non zero della matrice $\mathbf{A}$.*

4.3. Pseudocodice

In questo capitolo non presenteremo uno pseudocodice per ogni algoritmo iterativo. Ciascuno di questi metodi differisce dall'altro esclusivamente dal modo in cui viene calcolata l'iterata $\mathbf{x}^{(k)}$.

In Algoritmo 6 presentiamo la generica struttura di un algoritmo iterativo per il calcolo della soluzione di un sistema lineare.

Algorithm 6 Procedura iterativa generica.

Input: $A \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n, \text{tol};$

Output: $\mathbf{x} \in \mathbb{R}^n;$

```

1: inizializzo  $\mathbf{x}^{(0)} \in \mathbb{R}^n;$ 
2:  $k = 0;$ 
3: while CRITERIODIARRESTO( $\mathbf{x}^{(k)}$ ) do
4:    $\mathbf{x}^{(k+1)} = \text{UPDATE}(\mathbf{x}^{(k)});$ 
5:    $k = k + 1;$ 
6:   if  $k > \text{maxIter}$  then
7:     errore non converge
8:     break;
9:   end if
10: end while
11:  $\mathbf{x} = \mathbf{x}^{(k)};$ 

```

Un generico algoritmo per la risoluzione di un sistema lineare ha come input la matrice, A , il termine noto, $\mathbf{b}$, e la tolleranza tol . Vedremo che alcuni modi di calcolare le nuove iterate, $\text{UPDATE}(\mathbf{x}^{(k+1)})$, richiedono ulteriori parametri che dovranno essere messi come input.

All'inizio dell'algoritmo si deve considerare un primo vettore $\mathbf{x}^{(0)} \in \mathbb{R}^n$. Vedremo che sotto opportune ipotesi sulla matrice A , questa scelta non influenza la convergenza del processo iterativo. Ossia si può partire da un qualsiasi vettore $\mathbf{x}^{(0)} \in \mathbb{R}^n$ e, dopo un numero finito di passaggi, si arriva alla soluzione ricercata.

Il cuore dell'Algoritmo 6 è un ciclo **while** che ha come argomento una funzione, CRITERIODIARRESTO, che controlla uno dei criteri descritti in Sezione 4.2.

Dato che Algoritmo 6 potrebbe non terminare perché a priori non è detto che il criterio di arresto sia raggiunto, è sempre buona norma mettere un controllo sul numero di iterate eseguite. È per questo motivo che è stato inserito un **if** a linea 6 in Algoritmo 6.

Un'ultima osservazione importante su Algoritmo 6 è non vengono fatte operazioni che non alterano la struttura della matrice A . Infatti abbiamo visto che nella funzione CRITERIODIARRESTO vengono fatti prodotti matrice-vettore e somme di vettori. Vedremo poi nelle sezioni seguenti che anche le varie routine di UPDATE non si baseranno su scomposizioni ma operazioni base di prodotto e somma. Di conseguenza come potete immaginare per questo tipo di algoritmi non ci sarà il rischio di fill-in.

Nota Implementativa 4.3. Vedremo che per alcuni problemi il numero di iterazioni massime può essere stabilito a priori in base alla dimensione della matrice. In generale si considera sempre un numero arbitrariamente alto.

4.4. Metodi iterativi stazionari

Una strategia comune per definire un metodo risolutivo iterativo è tramite la procedura di splitting. Consideriamo una matrice $A \in \mathbb{R}^{n \times n}$ e supponiamo che si possa spezzare nel seguente modo

$$(4.3) \quad A = P - N,$$

dove $P, N \in \mathbb{R}^{n \times n}$. Grazie a questa decomposizione possiamo riscrivere il sistema lineare nel seguente modo

$$\begin{aligned} Ax &= b \\ (P - N)x &= b && \text{utilizzo Equazione (4.3)} \\ Px &= Nx + b \\ x &= P^{-1}Nx + P^{-1}b. \end{aligned}$$

Quest'ultima uguaglianza suggerisce una procedura iterativa. Infatti partendo da un primo tentativo di soluzione, i.e., $\mathbf{x}^{(0)} \in \mathbb{R}^n$ possiamo ricavare i successivi nel seguente modo

$$(4.4) \quad \mathbf{x}^{(k+1)} = P^{-1}N\mathbf{x}^{(k)} + P^{-1}\mathbf{b}.$$

A questo punto si potrebbe pensare che il problema sia complicato. L'idea che sta alla base dei metodi iterativi è che calcolare l'inversa di P deve essere facile e veloce. Infatti se trovare l'inversa di P fosse molto costoso, tanto vale trovare direttamente l'inversa della matrice A cosicché si possa calcolare subito la soluzione $\mathbf{x}$ del sistema, $\mathbf{x} = A^{-1}\mathbf{b}$.

Inoltre da Equazione (4.4) è chiaro perché si parla di metodi iterativi stazionari. Infatti il modo in cui viene calcolata l'iterata successiva non dipende dall'iterazione: le matrici P e N non dipendono dal parametro k .

I metodi iterativi che presenteremo in questa sezione si basano sulla decomposizione in Equazione (4.3). Per ognuno di essi diremo come costruire le matrici P e N , ma prima dobbiamo rispondere a questa domanda: è sempre possibile arrivare alla soluzione indipendentemente dal punto di partenza?

Proposizione 4.1. *Supponiamo che esista una decomposizione di A come in Equazione (4.3), con A e P simmetriche e definite positive. Se anche la matrice $2P - A$ è definita positiva allora il metodo iterativo definito in Equazione (4.4) converge per ogni valore iniziale $\mathbf{x}^{(0)}$ e*

$$|\lambda_{\max}| < 1,$$

dove $\lambda_{\max}$ è l'autovalore di modulo massimo della matrice $P^{-1}N$. Inoltre la convergenza è monotona rispetto alle norme $\|\cdot\|_p$.

Definizione 4.1. *La matrice $P^{-1}N$, che solitamente è indicata con B , viene detta matrice di iterazione del metodo.*

Proposizione 4.1 è molto importante per gli algoritmi iterativi basati sullo splitting. In primo luogo ci dice che sotto opportune ipotesi sulle matrici A e P , il processo iterativo converge sempre, ossia per ogni scelta di dato iniziale $\mathbf{x}^{(0)}$.

In secondo luogo la convergenza è monotona. Questo vuol dire che a ogni passaggio ci si avvicina sempre di più alla soluzione esatta e quindi l'errore decresce sempre.

Nell'espressione di Equazione (4.4) si possono manipolare le entrate in modo da far apparire un'espressione simile al residuo scalato, Equazione (4.2). Infatti,

$$\begin{aligned}
 \mathbf{x}^{(k+1)} &= P^{-1}N\mathbf{x}^{(k)} + P^{-1}\mathbf{b} \\
 \mathbf{x}^{(k+1)} &= P^{-1}(P - A)\mathbf{x}^{(k)} + P^{-1}\mathbf{b} && \text{definizione di } N = P - A \\
 \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - P^{-1}A\mathbf{x}^{(k)} + P^{-1}\mathbf{b} && P^{-1}P = I \\
 \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + P^{-1}(\mathbf{b} - A\mathbf{x}^{(k)}) \\
 (4.5) \quad \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)} && \text{definiamo } \mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}
 \end{aligned}$$

Il vettore $\mathbf{r}^{(k)}$ viene solitamente chiamato residuo. Computare questo vettore potrebbe essere utile se si sta utilizzando il criterio di arresto basato sul residuo scalato. Infatti $\mathbf{r}^{(k)}$ è il vettore che sta a numeratore del residuo scalato, i.e.,

$$(4.6) \quad \frac{\|A\mathbf{x}^{(k)} - \mathbf{b}\|}{\|\mathbf{b}\|} = \frac{\|\mathbf{r}^{(k)}\|}{\|\mathbf{b}\|}.$$

Nota Implementativa 4.4. In termini di costo in termini di operazioni è chiaro che, qualora utilizzassimo il criterio di arresto del residuo scalato, per calcolare il nuovo vettore $\mathbf{x}^{(k+1)}$, Equazione (4.5), calcoliamo già parte del criterio d'arresto, si confronti Equazione (4.5) e (4.6).

4.4.1. Jacobi. In questo paragrafo introduciamo il metodo iterativo stazionario di Jacobi. Consideriamo la i -esima equazione di un sistema lineare $A\mathbf{x} = \mathbf{b}$

$$a_{i,1}x_1 + \dots + a_{i,i-1}x_{i-1} + a_{i,i}x_i + a_{i,i+1}x_{i+1} + \dots + a_{i,n}x_n = b_i,$$

se risolvessimo questa equazione rispetto alla variabile x_i avremmo

$$x_i = \frac{1}{a_{i,i}} (b_i - a_{i,1}x_1 - \dots - a_{i,i-1}x_{i-1} - a_{i,i+1}x_{i+1} - \dots - a_{i,n}x_n).$$

Possiamo pensare di fare questa procedura per ogni entrata del vettore $\mathbf{x}$. Sfruttando la sommatoria possiamo scrivere come computare ognuna di queste entrate in forma compatta.

$$\begin{aligned}
 x_1 &= \frac{1}{a_{1,1}} \left(b_1 - \sum_{k \neq 1} a_{1,k}x_k \right), \\
 x_2 &= \frac{1}{a_{2,2}} \left(b_2 - \sum_{k \neq 2} a_{2,k}x_k \right), \\
 &\vdots \\
 x_i &= \frac{1}{a_{i,i}} \left(b_i - \sum_{k \neq i} a_{i,k}x_k \right), \\
 &\vdots \\
 x_n &= \frac{1}{a_{n,n}} \left(b_n - \sum_{k \neq n} a_{n,k}x_k \right).
 \end{aligned}
 \tag{4.7}$$

Un aspetto importante da evidenziare nelle equazioni precedenti sono gli indici delle sommatorie evidenziati in rosso. Per ogni equazione che si salta l'indice dell'entrata corrispondente. Infatti nella prima equazione, x_1 , la sommatoria considera indici $j \neq 1$, nella seconda, x_2 , la sommatoria ha $j \neq 2$, e così via fino all'ultima.

Partendo da Equazione (4.7), si possono definire le matrici P e N della matrice A e quindi Equazione (4.4) per il metodo di Jacobi. Consideriamo una generica matrice $A \in \mathbb{R}^{n \times n}$

$$A := \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix},$$

e definiamo le seguenti matrici per il metodo iterativo di Jacobi:

P : è la diagonale di A

$$(4.8) \quad P := \begin{bmatrix} a_{1,1} & 0 & \cdots & 0 \\ 0 & a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{n,n} \end{bmatrix}.$$

Questa matrice è facilmente invertibile. Infatti la sua inversa è una matrice diagonale che ha i coefficienti che sono i reciproci di quelli di P , ossia

$$P^{-1} := \begin{bmatrix} 1/a_{1,1} & 0 & \cdots & 0 \\ 0 & 1/a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/a_{n,n} \end{bmatrix}.$$

N : è la matrice che si ottiene mettendo a 0 le entrate sulla diagonale di A e considerando l'opposto di tutte le altre entrate

$$(4.9) \quad N := \begin{bmatrix} 0 & -a_{1,2} & \cdots & -a_{1,n} \\ -a_{2,1} & 0 & \cdots & -a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ -a_{n,1} & -a_{n,2} & \cdots & 0 \end{bmatrix}.$$

Si vede facilmente che le matrici P e N , definite rispettivamente in Equazione (4.8) e (4.9), soddisfano la decomposizione in Equazione (4.3).

Convergenza. Per dimostrare che il metodo iterativo di Jacobi converge dovremmo verificare le ipotesi della Proposizione 4.1. Esse però sono molto complicate dato che richiedono il calcolo dell'autovalore massimo della matrice di iterazione, $P^{-1}N$.

Tuttavia esistono casi in cui semplicemente guardando i coefficienti della matrice A è possibile stabilire se il metodo di Jacobi converge. Diamo la seguente definizione.

Definizione 4.2. Una matrice $A \in \mathbb{R}^{n \times n}$ è a dominanza diagonale stretta per righe se vale

$$|a_{i,i}| > \sum_{j=1, j \neq i}^n |a_{i,j}|.$$

In modo analogo $A \in \mathbb{R}^{n \times n}$ è a dominanza diagonale stretta per colonne se vale

$$|a_{i,i}| > \sum_{j=1, j \neq i}^n |a_{j,i}|.$$

La definizione dice in modo formale che l'elemento che sta sulla diagonale “domina”, ossia è il più grande in modulo rispetto a tutti gli altri che sono sulla stessa riga, è perfino più grande della somma di tutti gli altri termini che sono sulla diagonale.

Partendo da questa definizione abbiamo il seguente risultato più generale di Proposizione 4.1.

Teorema 4.1. *Se $A \in \mathbb{R}^{n \times n}$ è una matrice a dominanza diagonale stretta per righe il metodo di Jacobi converge.*

Osservazione 4.2. *Da notare che la condizione precedente è una condizione sufficiente ma non necessaria. Ossia potrebbero esserci matrici non a dominanza diagonale stretta per cui il metodo di Jacobi converge.*

Rapidità. In questo paragrafo calcoleremo il numero di operazioni macchina richieste per calcolare il vettore $\mathbf{x}^{(k+1)}$ durante una generica iterazione. A questo conto dovrà essere sommato il costo per la valutazione delle quantità del criterio di arresto, vedi Sezione 4.2.

Prima di valutare il costo di una iterata osserviamo che la matrice P^{-1} è la stessa per ogni iterazione. Di conseguenza possiamo computarla una volta sola all'inizio del processo iterativo. Dato che la matrice inversa di P si ottiene facendo il reciproco di ogni entrata sulla diagonale di A , si dovranno eseguire n divisioni e quindi il costo è $O(n)$.

Per il calcolo di ogni iterata del metodo di Jacobi si devono fare le seguenti operazioni:

- (1) calcolo del residuo che è composto dal costo del prodotto fra la matrice A e il vettore $\mathbf{x}^{(k)}$, $O(n^2)$, e la somma di due vettori, $O(n)$. Quindi avremmo

$$\underbrace{O(n^2)}_{A\mathbf{x}^{(k)}} + \underbrace{O(n)}_{\text{differenza}} = O(n^2).$$

- (2) il prodotto $P^{-1}\mathbf{r}^{(k)}$. Questo è un prodotto matrice vettore che sappiamo costare $O(n^2)$, tuttavia è possibile ridurre questo costo dato che la matrice P^{-1} è diagonale. Sfruttando la particolare struttura di P^{-1} vediamo che il costo è $O(n)$, infatti

$$P^{-1}\mathbf{r}^{(k)} = \begin{bmatrix} 1/a_{1,1} & 0 & \cdots & 0 \\ 0 & 1/a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1/a_{n,n} \end{bmatrix} \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_n \end{bmatrix} = \begin{bmatrix} r_1/a_{1,1} \\ r_2/a_{2,2} \\ \vdots \\ r_n/a_{n,n} \end{bmatrix},$$

quindi le operazioni che si devono fare sono semplicemente il prodotto delle componenti di $\mathbf{r}^{(k)}$ con le corrispondenti entrate sulla diagonale. In questo caso particolare si fanno n prodotti e di conseguenza il costo è $O(n)$.

- (3) la somma $\mathbf{x}^{(k)}$ con $P^{-1}\mathbf{r}^{(k)}$ che richiede $O(n)$.

In conclusione una iterata del metodo iterativo di Jacobi costa

$$\underbrace{O(n^2)}_{(1)} + \underbrace{O(n)}_{(2)} + \underbrace{O(n)}_{(3)} = O(n^2).$$

Nota Implementativa 4.5. *Come anticipato in precedenza il calcolo di $\mathbf{r}^{(k)}$ è funzionale per il calcolo del criterio di arresto basato sul residuo. È conveniente salvare questa variabile in modo da sfruttarla per il calcolo del residuo scalato, vedi Equazione (4.2).*

Osservazione 4.3. *Il costo del calcolo di P^{-1} è di ordine inferiore rispetto al costo di una sola iterata, $O(n)$ contro $O(n^2)$. Di conseguenza il maggior sforzo computazionale è dovuto al calcolo di $\mathbf{x}^{(k)}$ e non di P^{-1} .*

Lo pseudocodice per il calcolo della k -esima iterata di Jacobi si riduce a operazioni elementari fra matrici e vettori

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}(\mathbf{r}^{(k)}),$$

dove ricordiamo che

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}.$$

Metodo del rilassamento simultaneo con Jacobi (JOR). Partendo dal metodo di Jacobi è possibile creare un nuovo metodo iterativo. Consideriamo la formula generica per ottenere l' i -esima entrata del vettore $\mathbf{x}$

$$x_i = \frac{\omega}{a_{i,i}} \left(b_i - \sum_{k \neq i} a_{i,k} x_k \right) + (1 - \omega) x_i,$$

dove $\omega \in \mathbb{R}$ è un parametro che deve essere opportunamente scelto. In particolare osserviamo che se $\omega = 1$, abbiamo esattamente il metodo di Jacobi, i.e., le formule di Equazione (4.7).

L'introduzione di questo parametro porta alla seguente modifica della matrice P ,

$$P^{-1} := \begin{bmatrix} \omega/a_{1,1} & 0 & \cdots & 0 \\ 0 & \omega/a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \omega/a_{n,n} \end{bmatrix}.$$

Osservazione 4.4. La matrice P^{-1} del metodo di Jacobi con rilassamento è il prodotto fra la matrice P di Jacobi senza rilassamento per il fattore ω . In formule abbiamo

$$P_{JR}^{-1} = \omega P_J^{-1},$$

dove P_J^{-1} è la matrice del metodo di Jacobi definita in Equazione (4.8).

La velocità ma anche la convergenza del metodo di Jacobi è influenzata dalla presenza di questo parametro ω . In particolare vale il seguente teorema che offre un risultato di convergenza solamente per il metodo di Jacobi.

Teorema 4.2. Se il metodo di Jacobi converge allora il metodo di Jacobi con rilassamento simultaneo converge per valori di $\omega \in (0, 1]$.

Il metodo di Jacobi e la sua versione con il rilassamento simultaneo sono equivalenti sotto l'aspetto pratico. Ossia il costo in termini di operazione macchina per calcolare una iterazione rimane $O(n^2)$. Inoltre il suo pseudocodice comporterà solamente la modifica della matrice P utilizzata.

4.4.2. Gauß-Seidel. Il metodo di Gauß-Seidel è una variante del metodo di Jacobi. Vediamo come viene calcolata l'entrata i -esima del vettore $\mathbf{x}$

$$(4.10) \quad x_i^{(k+1)} = \frac{1}{a_{i,i}} \left(b_i - a_{i,1} x_1^{(k)} - \dots - a_{i,i-1} x_{i-1}^{(k)} - a_{i,i+1} x_{i+1}^{(k)} - \dots - a_{i,n} x_n^{(k)} \right).$$

Da questa equazione notiamo che nel termine di destra ci sono tutti valori che dipendono dall'iterata precedente. L'idea su cui si basa è molto semplice: sfruttare le entrate del vettore $\mathbf{x}$ già calcolate durante l'iterazione attuale. Se dovessimo scrivere la formula, l'Equazione (4.10) diventerà

$$(4.11) \quad x_i^{(k+1)} = \frac{1}{a_{i,i}} \left(b_i - a_{i,1} x_1^{(k+1)} - \dots - a_{i,i-1} x_{i-1}^{(k+1)} - a_{i,i+1} x_{i+1}^{(k)} - \dots - a_{i,n} x_n^{(k)} \right).$$

da questa equazione si può notare che il calcolo di $x_i^{(k+1)}$ dipende dalle entrate del vettore precedente, $x_{i+1}^{(k)}, \dots, x_n^{(k)}$, e dalle entrate appena calcolate, $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$.

Proprio come abbiamo fatto per il metodo di Jacobi, consideriamo una generica matrice quadrata $A \in \mathbb{R}^{n \times n}$ e scriviamo quali sono le matrici P e N del metodo di Gauß-Seidel:

$\mathbf{P}$: è la parte triangolare inferiore della matrice A

$$(4.12) \quad P := \begin{bmatrix} a_{1,1} & 0 & \cdots & 0 \\ a_{2,1} & a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix}.$$

$\mathbf{N}$: è la parte triangolare superiore della matrice A con segno opposto e priva dei termini sulla diagonale principale

$$(4.13) \quad N := \begin{bmatrix} 0 & -a_{1,2} & \cdots & -a_{1,n} \\ 0 & 0 & \cdots & -a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}.$$

Si vede facilmente che le matrici P e N , definite rispettivamente in Equazione (4.12) e (4.13), soddisfano la decomposizione in Equazione (4.3).

Una delle caratteristiche di un metodo iterativo è che la matrice P sia facilmente invertibile. Tuttavia non è possibile trovare la matrice inversa di una triangolare superiore. Rimane allora aperta la domanda: come è possibile calcolare

$$(4.14) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)},$$

senza passare attraverso il calcolo della matrice inversa?

Quando si trattano problemi legati all'algebra lineare in numerica, ci si trova spesso di fronte a espressioni come quella di Equazione (4.14), ma si evita sempre il calcolo dell'inversa dato che è molto costoso e poco stabile numericamente.

Per sopperire a questo problema si cambia prospettiva e si guarda l'azione che la matrice ha nelle operazioni. In particolare, come nel caso di Equazione (4.14), possiamo pensare a $P^{-1}\mathbf{r}^{(k)}$ come la soluzione di un sistema lineare. Infatti, trovare la soluzione $\mathbf{y}$ del sistema lineare $P\mathbf{y} = \mathbf{r}^{(k)}$ equivale ad avere direttamente risultato del prodotto $P^{-1}\mathbf{r}^{(k)}$, in formule

$$\begin{array}{ll} P\mathbf{y} = \mathbf{r}^{(k)} & \text{ sistema fittizio di partenza} \\ P^{-1}P\mathbf{y} = P^{-1}\mathbf{r}^{(k)} & \text{ moltiplico per } P^{-1} \\ \mathbf{y} = P^{-1}\mathbf{r}^{(k)} & \text{ abbiamo direttamente } P^{-1}\mathbf{r}^{(k)}. \end{array}$$

Anziché trovare P^{-1} ci siamo ricondotti a risolvere un sistema lineare. Dato che la matrice P è triangolare inferiore, la risoluzione di tale sistema lineare è veloce. Infatti, è sufficiente applicare la sostituzione in avanti e non svolgere ulteriori operazioni o decomposizioni.

Convergenza. Oltre al risultato di Proposizione 4.1 anche per il metodo iterativo di Gauß-Seidel vale un risultato analogo a quello di Teorema 4.1.

Teorema 4.3. *Se $A \in \mathbb{R}^{n \times n}$ è una matrice a dominanza diagonale stretta per righe il metodo di Gauß-Seidel converge.*

Osservazione 4.5. *Anche in questo caso la condizione di convergenza è sufficiente ma non necessaria. Di conseguenza potrebbero esserci matrici non a dominanza diagonale stretta per cui il metodo di Gauß-Seidel converge.*

Rapidità. Vediamo il costo in termini di operazioni macchina per il calcolo di una generica iterata del metodo di Gauß-Seidel. Ricordiamo che per valutare il costo complessivo della

risoluzione di un sistema lineare con il metodo di Gauß-Seidel si dovrà sommare il costo per il criterio di arresto e quante iterate devono essere fatte.

Consideriamo anche in questo caso la seguente formula di aggiornamento del vettore

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}.$$

Essa richiederà

- (1) calcolo del residuo che è composto dal costo del prodotto fra la matrice A e il vettore $\mathbf{x}^{(k)}$, $O(n^2)$, e la somma di due vettori, $O(n)$. Di conseguenza, proprio come per Jacobi, avremmo un costo di $O(n^2)$ per il calcolo del residuo.
- (2) il prodotto $P^{-1}\mathbf{r}^{(k)}$ viene interpretato come risoluzione di un sistema lineare, dato che invertire la matrice triangolare inferiore è un'operazione molto onerosa. Come si è visto prima, invece che procedere con l'inversione della matrice, si calcola la soluzione, $\mathbf{y}$, del sistema lineare

$$P\mathbf{y} = \mathbf{r}^{(k)} \Rightarrow \mathbf{y} = P^{-1}\mathbf{r}^{(k)}.$$

Dato che il sistema è triangolare inferiore questa operazione costa $O(n^2)$, vedi Sezione 3.1.

- (3) la somma $\mathbf{x}^{(k)}$ con $P^{-1}\mathbf{r}^{(k)}$ che richiede $O(n)$.

In conclusione una iterata del metodo iterativo di Gauß-Seidel costa

$$\underbrace{O(n^2)}_{(1)} + \underbrace{O(n^2)}_{(2)} + \underbrace{O(n)}_{(3)} = O(n^2).$$

A differenza di prima il costo maggiore è da attribuire sia al residuo sia alla risoluzione del sistema lineare, ma in entrambi i casi abbiamo $O(n^2)$ operazioni e di conseguenza lo sforzo computazionale rimane limitato.

Pseudocodice. Anche in questo caso lo pseudocodice dell'iterata k -esima di questo processo iterativo è molto semplice. Infatti esso è formato da tre istruzioni in sequenza:

Algorithm 7 Procedura per Gauß-Seidel.

Input: $A, P \in \mathbb{R}^{n \times n}, \mathbf{b}, \mathbf{x}^{(k)} \in \mathbb{R}^n$;

Output: $\mathbf{x}^{(k+1)}, \mathbf{r}^{(k)} \in \mathbb{R}^n$;

- 1: $\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$;
 - 2: sostituzione in avanti $P\mathbf{y} = \mathbf{r}^{(k)}$;
 - 3: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{y}$;
-

Una nota importante è che esso chiama al suo interno la procedura di sostituzione in avanti per evitare l'onerosa computazione della matrice P^{-1} , linea 2 di Algoritmo 7.

Da notare inoltre che l'output non è solamente la nuova iterata $\mathbf{x}^{(k+1)}$ ma anche il residuo, $\mathbf{r}^{(k)}$. Questo output può essere sfruttato per evitare la computazione del residuo nel criterio di arresto.

Nota Implementativa 4.6. Lo pseudocodice presentato in Algoritmo 7 vuole essere una sorta di riassunto delle operazioni da eseguire per calcolare l'iterata $\mathbf{x}^{(k+1)}$ di una procedura iterativa di Gauß-Seidel. Se viene messo all'interno della generica procedura di un processo iterativo può essere migliorato. Una miglioria potrebbe essere quella di calcolare il residuo all'esterno e darlo come input a Algoritmo 7.

Metodo del rilassamento simultaneo con Gauß-Seidel (SOR). Esattamente come abbiamo fatto per il metodo di Jacobi anche nel caso del metodo iterativo di Gauß-Seidel, è possibile introdurre la sua versione con il rilassamento simultaneo.

In questo caso l'entrata i -esima del vettore $\mathbf{x}$ verrà aggiornata nel seguente modo

$$x_i^{(k+1)} = \frac{\omega}{a_{i,i}} \left(b_i - \sum_{j < i} a_{i,j} x_j^{(k+1)} - \sum_{j > i} a_{i,j} x_j^{(k)} \right) + (1 - \omega) x_i^{(k)}.$$

La matrice P che verrà utilizzata per aggiornare il vettore $\mathbf{x}^{(k+1)}$ avrà la seguente forma

$$P := \begin{bmatrix} a_{1,1}/\omega & 0 & \cdots & 0 \\ a_{2,1} & a_{2,2}/\omega & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n}/\omega \end{bmatrix}.$$

Dato che la matrice è triangolare inferiore e il coefficiente ω influenza cambia solamente la diagonale della matrice P , nel caso del metodo di rilassamento di Gauß-Seidel, non vale una semplificazione simile a quella di Osservazione 4.4. Quindi, per implementare questo metodo, si deve cambiare la matrice P e non si può semplicemente prendere la matrice del metodo di Gauß-Seidel e moltiplicarla per ω .

Naturalmente anche in questo caso la presenza di questo nuovo parametro influenza la convergenza del metodo iterativo. In particolare vale la seguente teorema.

Teorema 4.4. *Se $A \in \mathbb{R}^{n \times n}$ è una matrice simmetrica e definita positiva, il metodo di rilassamento di Gauß-Seidel converge se e solo se $0 < \omega < 2$.*

Il metodo di Gauß-Seidel con e senza rilassamento simultaneo sono equivalenti sotto l'aspetto pratico. Se ci si pensa bene l'unica cosa che cambia è la matrice P : tutte le altre considerazioni riguardanti il numero di operazioni richieste e lo pseudocodice rimangono simili, quindi non verranno ripetute.

4.4.3. Metodi di Richardson. In tutti i metodi iterativi stazionari descritti in precedenza abbiamo considerato la seguente forma generale per calcolare l'iterata successiva $\mathbf{x}^{(k+1)}$, in formule

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1} \mathbf{r}^{(k)}.$$

I metodi che descriveremo in questo paragrafo modificano questa formula di aggiornamento introducendo un parametro reale α , i.e.,

$$(4.15) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha P^{-1} \mathbf{r}^{(k)},$$

tali metodi vengono chiamati metodi di Richardson.

Come si può vedere da Equazione (4.15) essi non si occupano di determinare la matrice P , ma aggiungono semplicemente il parametro reale α nella fase di aggiornamento del vettore $\mathbf{x}^{(k+1)}$.

In particolare si parla di metodi di Richardson basati sulla decomposizione di Jacobi, se si considera la matrice P definita in Equazione (4.8), oppure si parla di metodi di Richardson basati su Gauß-Seidel, se si considera invece la matrice P di Equazione (4.12).

La presenza di questo parametro influenza la convergenza del metodo. In particolare abbiamo questo risultato che permette di trovare il parametro ottimale, i.e., quello che diminuisce il più possibile le iterazioni richieste per la convergenza.

Teorema 4.5. Supponiamo che $P \in \mathbb{R}^{n \times n}$ sia una matrice non singolare e che $P^{-1}A \in \mathbb{R}^{n \times n}$ abbia autovalori reali e positivi. Ordiniamo questi autovalori da quello di modulo massimo a quello di modulo minimo:

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \lambda_n > 0.$$

Allora il metodo di Richardson converge se e solo se $0 < \alpha < 2/\lambda_1$. Inoltre il valore ottimale di questo parametro, α_{opt} , ossia il valore di α che minimizza le iterazioni è dato da

$$\alpha_{opt} = \frac{2}{\lambda_1 + \lambda_n}.$$

Il teorema precedente si basa sul calcolo degli autovalori di $P^{-1}A$. Data una matrice, il calcolo degli autovalori è un problema numerico a se stante che potrebbe richiedere un notevole sforzo computazionale al crescere delle dimensioni delle matrici.

I metodi di Richardson sono basati sui metodi di Jacobi e Gauß-Seidel, quindi gli aspetti pratici come rapidità e la loro implementazione sono del tutto analoghi a quanto detto precedentemente.

In particolare, per quanto riguarda il costo in termini di operazioni macchina, i metodi di Richardson basati su Jacobi e Gauß-Seidel hanno un costo di $O(n^2)$.

Sotto l'aspetto implementativo il codice dovrà essere modificato solamente nella parte in cui viene aggiornato il vettore $\mathbf{x}^{(k+1)}$.

4.5. Metodi iterativi non stazionari

Nella sezione precedente abbiamo costruito una sequenza di vettori $\mathbf{x}^{(k)}$ partendo dalla formula

$$(4.16) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha P^{-1} \mathbf{r}^{(k)},$$

i metodi che abbiamo descritto corrispondono a una scelta particolare dello scalare α e della matrice P . In questa sezione ci occuperemo di metodi che aggiornano il vettore $\mathbf{x}^{(k+1)}$ nel modo seguente

$$(4.17) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k P^{-1} \mathbf{r}^{(k)}.$$

La sostanziale differenza fra Equazione (4.16) e (4.17) sta nel coefficiente α . In tutti i metodi che abbiamo visto prima questo valore è fisso per qualsiasi iterazione, nei metodi non stazionari α dipende dalle iterazioni. Tale dipendenza dalle iterazioni è evidenziata dal pedice k .

4.5.1. Metodo del gradiente. Questo metodo si basa su un modo nuovo di interpretare la risoluzione di un sistema lineare. Presa un sistema lineare generico

$$(4.18) \quad A\mathbf{x} = \mathbf{b},$$

dove $A \in \mathbb{R}^{n \times n}$ e $\mathbf{b} \in \mathbb{R}^n$ è un vettore, definiamo la funzione $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ come

$$(4.19) \quad \phi(\mathbf{y}) := \frac{1}{2} \mathbf{y}^t A \mathbf{y} - \mathbf{b}^t \mathbf{y}.$$

Se supponiamo che la matrice A sia simmetrica e definita positiva, la funzione ϕ è una forma quadratica nella variabile vettoriale $\mathbf{y} \in \mathbb{R}^n$, ossia è l'analogo multidimensionale di una parabola che ha concavità verso l'alto.

Vedremo che trovare la soluzione del sistema lineare di Equazione (4.18), equivale a trovare l'unico punto di minimo di questa forma lineare multidimensionale. Proseguendo con l'analogia con il caso monodimensionale della parabola, la soluzione del sistema di Equazione (4.18) coincide con le coordinate del vertice.

Per trovare il minimo di ϕ , si considera il gradiente

$$\begin{aligned}
 \nabla\phi(\mathbf{y}) &= \frac{1}{2}(A^t + A)\mathbf{y} - \mathbf{b} && \text{definizione di derivata} \\
 &= \frac{1}{2}A^t\mathbf{y} + \frac{1}{2}A\mathbf{y} - \mathbf{b} \\
 &= \frac{1}{2}A\mathbf{y} + \frac{1}{2}A\mathbf{y} - \mathbf{b} && A \text{ è simmetrica} \\
 &= A\mathbf{y} - \mathbf{b}
 \end{aligned}$$

e si trova il vettore $\mathbf{y} \in \mathbb{R}^n$ tale per cui

$$A\mathbf{y} - \mathbf{b} = \mathbf{0},$$

che equivale a trovare la soluzione del sistema di partenza, vedi Equazione (4.18). Da notare che l'esistenza e l'unicità di tale vettore è dovuta alle ipotesi che abbiamo fatto sulla matrice A , in altri casi questa procedura non sarebbe possibile.

Sotto queste ipotesi sulla matrice A possiamo interpretare la ricerca della soluzione di un sistema lineare come un processo di ricerca di minimo della funzione ϕ . Pensare che la successione di vettori $\mathbf{x}^{(k)}$, non sono nient'altro che punti in $\mathbb{R}^n$ che si avvicinano sempre di più al minimo di ϕ .

In dimensione $n = 2$ è possibile “vedere” graficamente questa interpretazione della soluzione di un sistema lineare. Consideriamo allora una matrice $A \in \mathbb{R}^{2 \times 2}$ simmetrica e definita positiva e un vettore $\mathbf{b} \in \mathbb{R}^2$. In questo caso la funzione $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}$ è una superficie chiamata paraboloide, una sorta di calice rivolto verso l'alto, vedi Figura 4. La soluzione del sistema lineare $A\mathbf{x} = \mathbf{b}$ è il punto di minimo di questa superficie, ossia la coppia di coordinate (x_1, x_2) che danno il minimo di ϕ .

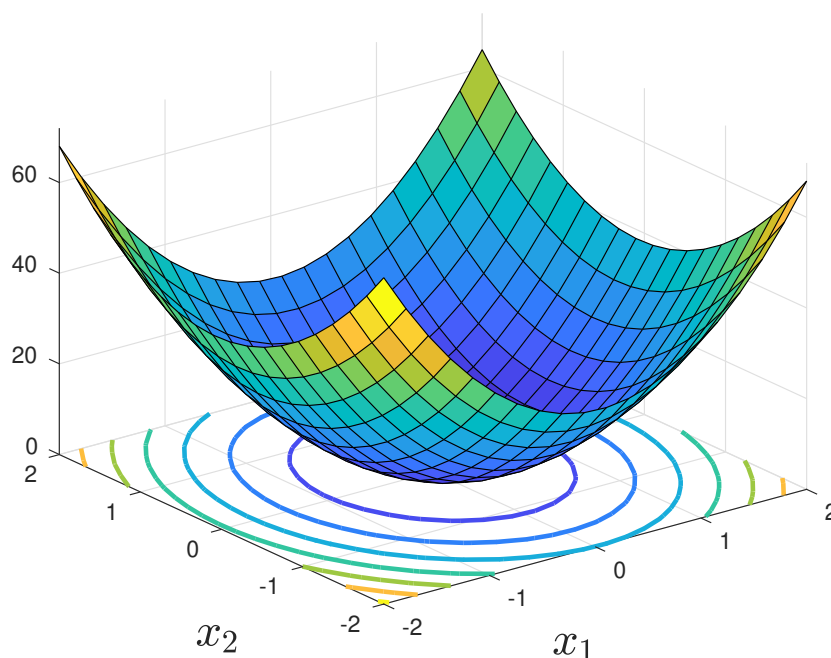


Figura 4. Rappresentazione grafica della funzione ϕ quando si considera una matrice $A \in \mathbb{R}^{2 \times 2}$, nel piano $x_1 O x_2$ sono messe le linee di livello.

Di conseguenza, possiamo interpretare la successione di vettori $\mathbf{x}^{(k)}$ come una “camminata” verso il punto di minimo di ϕ . Inoltre anche la formula con cui si calcola l’iterata al passo successivo può essere vista nel seguente modo

$$(4.20) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)},$$

dove $\mathbf{d}^{(k)}$ è la direzione verso cui mi muovo e α_k rappresenta l’intensità del movimento, vedi Figura 5.

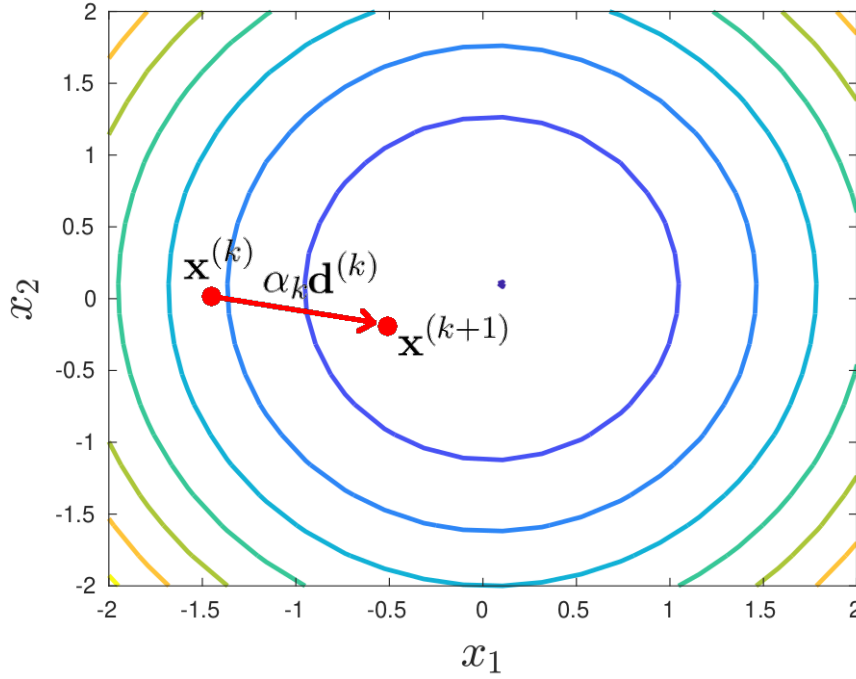


Figura 5. Rappresentazione grafica della funzione della successione di valori di $\mathbf{x}^{(k)}$ nel caso in cui si considera la matrice $A \in \mathbb{R}^{2 \times 2}$. La rappresentazione della superficie è fatta tramite linee di livello per poter notare meglio l’interpretazione geometrica di α_k e $\mathbf{d}^{(k)}$.

Dato che si vuole andare verso il minimo della funzione ϕ , l’idea è quella di prendere come direzione quella opposta al gradiente. Infatti, il gradiente della funzione valutato in un punto restituisce la direzione di massima crescita, la direzione opposta corrisponde quindi alla massima decrescita. Per questa caratteristica, i metodi che utilizzano la direzione opposta al gradiente vengono denominati massima discesa, in inglese steepest-descend. Fissato $\mathbf{x}^{(k)}$ la direzione di massima decrescita è data da

$$-\nabla\phi(\mathbf{x}^{(k)}) = -A\mathbf{x}^{(k)} + \mathbf{b}.$$

Tale direzione coincide con il residuo al passo k , i.e.,

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}.$$

Quindi la formula per l’aggiornamento della variabile $\mathbf{x}^{(k)}$, Equazione (4.20), diventerà

$$(4.21) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Occupiamoci ora della valore di α_k . Facciamo variare questo valore e introduciamo quindi una variabile α

$$\phi(\mathbf{x}^{(k+1)}) = \phi(\alpha) = \frac{1}{2}(\mathbf{x}^{(k)} + \alpha \mathbf{r}^{(k)})^t A (\mathbf{x}^{(k)} + \alpha \mathbf{r}^{(k)}) - (\mathbf{x}^{(k)} + \alpha \mathbf{r}^{(k)})^t \mathbf{b},$$

dove abbiamo semplicemente sostituito l'espressione per $\mathbf{x}^{(k+1)}$ in Equazione (4.21). Questa è una funzione quadratica nella variabile α e sappiamo che il suo minimo si ottiene annullando la sua derivata rispetto a α , i.e.,

$$\begin{aligned}\frac{d}{d\alpha}\phi(\alpha) &= \alpha \left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)} + \left(\mathbf{x}^{(k)}\right)^t A \mathbf{r}^{(k)} - \left(\mathbf{r}^{(k)}\right)^t \mathbf{b} && \text{definizione di derivata} \\ &= \alpha \left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)} + \left(\mathbf{r}^{(k)}\right)^t A \mathbf{x}^{(k)} - \left(\mathbf{r}^{(k)}\right)^t \mathbf{b} \\ &= \alpha \left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)} - \left(\mathbf{r}^{(k)}\right)^t (\mathbf{b} - A \mathbf{x}^{(k)}) \\ &= \alpha \left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)} - \left(\mathbf{r}^{(k)}\right)^t \mathbf{r}^{(k)} && \text{definizione di residuo}\end{aligned}$$

quindi abbiamo

$$\alpha \left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)} - \left(\mathbf{r}^{(k)}\right)^t \mathbf{r}^{(k)} = 0 \quad \Rightarrow \quad \alpha_k = \frac{\left(\mathbf{r}^{(k)}\right)^t \mathbf{r}^{(k)}}{\left(\mathbf{r}^{(k)}\right)^t A \mathbf{r}^{(k)}}.$$

Da notare che il coefficiente α_k cambia a ogni iterazione e che dipende solamente dal residuo e dalla matrice A .

Convergenza. In questo paragrafo ci occuperemo della convergenza del metodo del gradiente. Vale un risultato analogo a quello che abbiamo definito per i metodi stazionari.

Teorema 4.6. *Sia $A \in \mathbb{R}^{n \times n}$ una matrice simmetrica e definita positiva, allora il metodo del gradiente converge per ogni valore del dato iniziale $\mathbf{x}^{(0)}$.*

Dalla discussione fatta nel paragrafo precedente si può apprezzare quanto sia importante, o per meglio dire essenziale, l'ipotesi sulla matrice A . Se infatti A non fosse simmetrica e definita positiva, la funzione ϕ non sarebbe una funzione quadratica e di conseguenza tutte le ipotesi e ragionamenti fatti non sarebbero più validi.

La velocità della convergenza, ossia le iterazioni richieste per arrivare alla soluzione, dipende ancora numero di condizionamento della matrice che rappresenta il sistema lineare.

Questa affermazione si può dimostrare in modo rigoroso. Se si considerassero sistemi lineari di due equazioni in due incognite, è possibile dare anche una dimostrazione più intuitiva.

Ricordiamo che il numero di condizionamento di una matrice è il rapporto fra l'autovalore di modulo massimo e quello di modulo minimo. Nel caso preso in esame, una matrice $A \in \mathbb{R}^{2 \times 2}$ simmetrica e definita positiva, abbiamo due autovalori reali e positivi, quindi

$$\text{cond}(A) = \frac{\lambda_{\max}}{\lambda_{\min}},$$

dove $\lambda_{\max}$ e $\lambda_{\min}$ sono rispettivamente l'autovalore più grande e quello più piccolo. Si omettono in questo caso i moduli visto che sappiamo che sono positivi. Si può osservare che più questi autovalori hanno la medesima grandezza, più le linee di livello della funzione ϕ sono simili a cerchi. In Figura 6 riportiamo questi due casi.

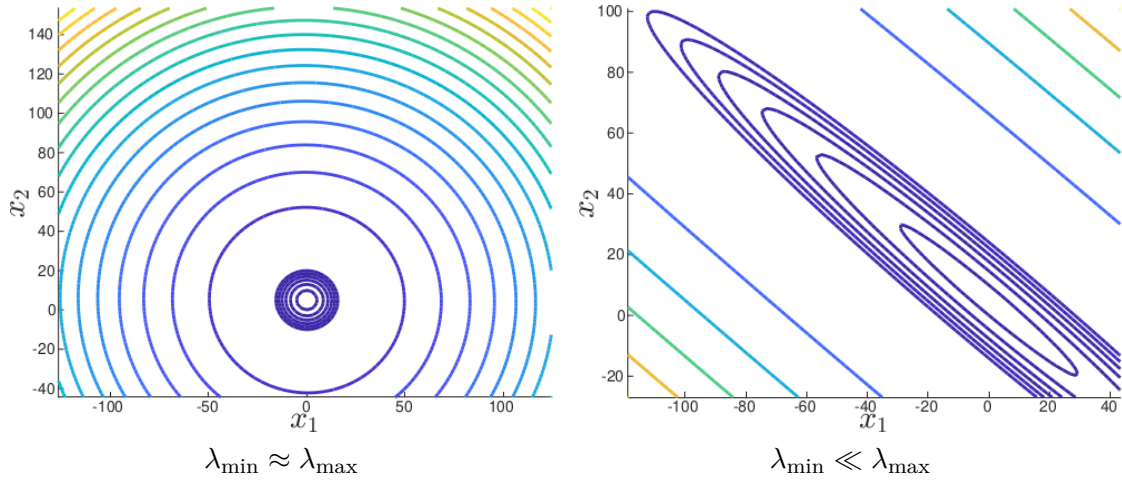


Figura 6. Linee di livello per due funzioni della forma di Equazione (4.19) definite da due matrici $A_1, A_2 \in \mathbb{R}^{2 \times 2}$ dove gli autovalori sono vicini, $\lambda_{\min} \approx \lambda_{\max}$, oppure molto diversi, $\lambda_{\min} \ll \lambda_{\max}$.

Ricordiamo che la coppia $\bar{x}_1$ e $\bar{x}_2$ che realizza il minimo è al centro di questi “bersagli”. Il metodo del gradiente crea una sequenza di vettori $\mathbf{x}^{(k)}$ le cui componenti convergeranno rispettivamente ai valori $\bar{x}_1$ e $\bar{x}_2$.

In Figura 7 riportiamo la sequenza creata dal metodo del gradiente in entrambi i casi. Come si può vedere la forma delle linee di livello influenza il numero di iterazioni necessarie ad arrivare al punto di minimo. In particolare più gli autovalori sono simili, $\lambda_{\min} \approx \lambda_{\max}$, servono poche iterazioni per la convergenza. Si veda la Figura 7 a sinistra: il metodo del gradiente richiede 7 iterazioni per la convergenza.

Nel caso $\lambda_{\min} \ll \lambda_{\max}$, la convergenza non è diretta ma procede a zig-zag. Infatti, come si può vedere da Figura 7 a destra, la procedura iterativa non va direttamente al centro come prima, ma servono molte più iterate: 147 iterazioni per arrivare a convergenza.

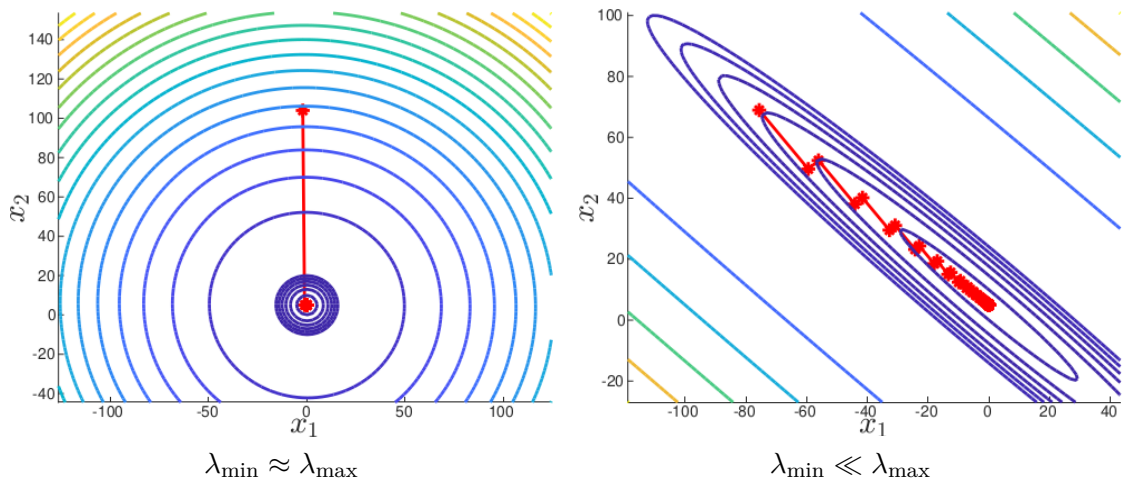


Figura 7. Iterazioni richieste dal metodo del gradiente per andare a convergenza per le funzioni ϕ di Figura 6.

Oltre alla forma influenza molto anche il punto di partenza. Infatti anche nel caso di autovalori $\lambda_{\min} \ll \lambda_{\max}$ possiamo diminuire molto il numero di iterazioni. In Figura 8 consideriamo un altro caso e due diversi punti di partenza. Nel primo caso abbiamo un andamento a zig-zag

simile a quello visto in precedenza: le iterazioni per convergere alla soluzione sono 202. Nel secondo caso, partendo da un altro punto le iterazioni diventano molto meno, 6.

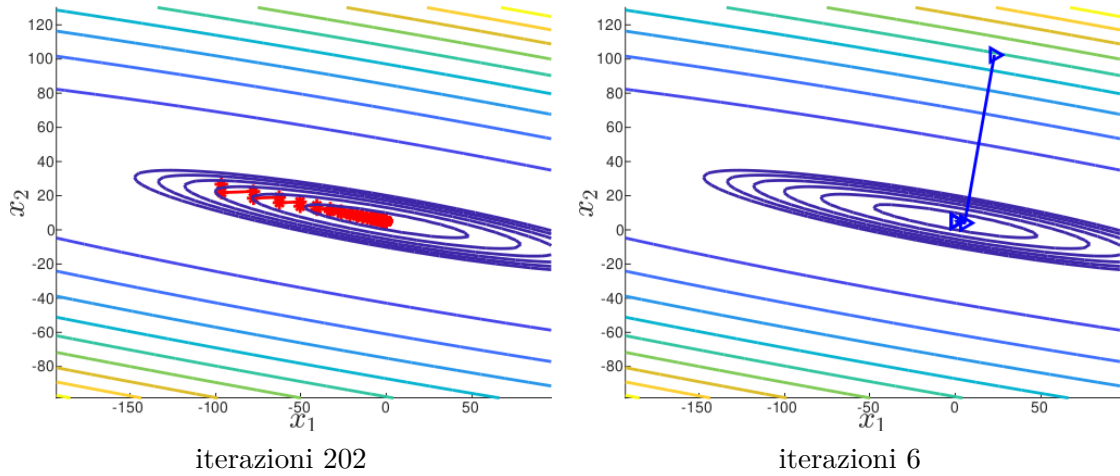


Figura 8. Iterazioni richieste dal metodo del gradiente per andare a convergenza per una funzione ϕ la cui matrice A ha autovalori $\lambda_{\min} \ll \lambda_{\max}$. Sebbene gli autovalori siano molto diversi, si riesce a contenere il numero di iterazioni partendo da punti diversi.

Nel caso in cui gli autovalori siano simili, il punto di partenza non influenza il numero di iterazioni per arrivare a convergenza. Rifacciamo l'esempio precedente considerando il caso di autovalori simili, $\lambda_{\min} \approx \lambda_{\max}$. In questo caso abbiamo la convergenza sempre in 4 iterazioni e si vede come la successione costruita punti sempre subito verso la coppia x_1 e x_2 , riportiamo i risultati in Figura 9.

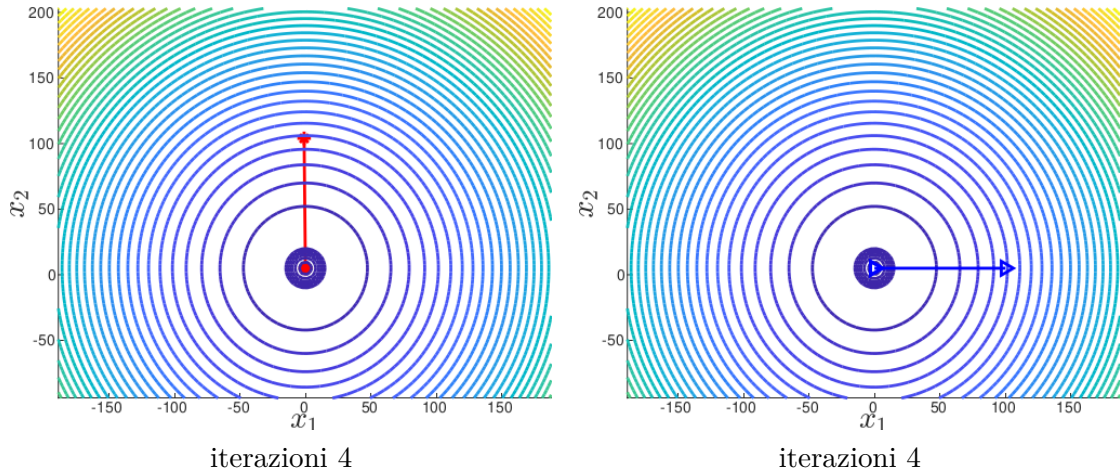


Figura 9. Iterazioni richieste dal metodo del gradiente per andare a convergenza per una funzione ϕ la cui matrice A ha autovalori $\lambda_{\min} \approx \lambda_{\max}$. Il numero di iterazioni è simile partendo da punti diversi.

In questo ambito non si cerca il punto di partenza ottimale per diminuire il numero di iterazioni. Si predilige il fatto di modificare la matrice A in modo da rendere le curve di livello più simili possibili a circonferenze. Tali tecniche prendono il nome di tecniche di preconditionamento e mirano a trovare quella matrice P che modifica gli autovalori di A in modo tale che il rapporto

$$\frac{|\lambda_{\max}|}{|\lambda_{\min}|} \approx 1.$$

Rapidità. Calcoliamo lo sforzo in termini di operazioni macchina per questo metodo durante una generica iterazione k . Preso un vettore $\mathbf{x}^{(k)}$ per trovare il valore successivo dobbiamo computare

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Ora vediamo il costo per ottenere ogni singolo pezzo e poi vediamo il costo totale di una iterazione:

- (1) calcolo di $\mathbf{r}^{(k)}$ abbiamo già visto che questo conto costa di un prodotto matrice-vettore e una somma fra vettori, quindi il costo è $O(n^2)$.
- (2) il calcolo di

$$\alpha_k := \frac{(\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^t A \mathbf{r}^{(k)}}$$

richiede il prodotto scalare del vettore $\mathbf{r}^{(k)}$ per il numeratore, $O(n)$. Per quanto riguarda il denominatore lo possiamo spezzare in due parti. Prima viene fatto il prodotto matrice-vettore, fra A e $\mathbf{r}^{(k)}$,

$$\mathbf{y}^{(k)} = A \mathbf{r}^{(k)},$$

che costa $O(n^2)$ operazioni. Poi verrà fatto il prodotto scalare fra il vettore $\mathbf{y}^{(k)}$ e $\mathbf{r}^{(k)}$, $O(n)$ operazioni. Quindi per calcolare α_k sono richieste

$$\underbrace{O(n)}_{(\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}} + \underbrace{O(n^2)}_{A \mathbf{r}^{(k)}} + \underbrace{O(n)}_{(\mathbf{r}^{(k)})^t \mathbf{y}^{(k)}} + \underbrace{1}_{\text{divisione}} = O(n^2).$$

operazioni macchina.

- (3) il prodotto fra lo scalare α_k e tutte le entrate del vettore $\mathbf{r}^{(k)}$, $O(n)$.
- (4) la somma fra il vettore $\mathbf{x}^{(k)}$ e $\alpha_k \mathbf{r}^{(k)}$, $O(n)$ operazioni macchina.

Quindi sommando tutti questi contributi avremmo

$$\underbrace{O(n^2)}_{(1)} + \underbrace{O(n^2)}_{(2)} + \underbrace{O(n)}_{(3)} + \underbrace{O(n)}_{(4)} = O(n^2).$$

Nota Implementativa 4.7. Tutte le operazioni precedenti per il calcolo di una iterata sono operazioni base fra matrici e vettori. Le più comuni librerie sfruttano la struttura sparsa della matrice A per ottimizzarle e renderle più veloci, anche sfruttando la struttura multicore dei processori. Quindi a livello di codice è sempre meglio utilizzare le operazioni implementate all'interno della libreria e non fare uso pesante di cicli `for`.

Nota Implementativa 4.8. Un altro vantaggio è quello che se si utilizzano gli operatori di prodotto e somma di matrici vettori il codice diventa anche più leggibile e “simile” a quello che si fa su carta quindi anche più semplice da debuggare.

Pseudocodice. Presentiamo lo pseudocodice per calcolare la k -esima iterata del metodo del gradiente. Anche in questo caso le istruzioni si riducono a una sequenza di operazioni algebriche base.

Algorithm 8 Procedura per il metodo del gradiente.

Input: $A \in \mathbb{R}^{n \times n}$, $\mathbf{x}^{(k)} \in \mathbb{R}^n$;
Output: $\mathbf{x}^{(k+1)}$, $\mathbf{r}^{(k)} \in \mathbb{R}^n$;

- 1: $\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$;
 - 2: $\mathbf{y}^{(k)} = A\mathbf{r}^{(k)}$;
 - 3: $a = (\mathbf{r}^{(k)})^t \mathbf{r}^{(k)}$;
 - 4: $b = (\mathbf{r}^{(k)})^t \mathbf{y}^{(k)}$;
 - 5: $\alpha_k = a/b$;
 - 6: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}$;
-

Anche in questo caso gli input e output della funzione che calcola l'iterata dipende dalla struttura in cui questa procedura è inserita. Per esempio il residuo, $\mathbf{r}^{(k)}$, potrebbe essere stato precedentemente calcolato e quindi essere preso come input.

4.5.2. Metodo del gradiente coniugato. In questa sezione parleremo del metodo del gradiente coniugato. Questo metodo può essere visto come un miglioramento del metodo del gradiente dato che pone rimedio all'effetto di “convergenza a zig-zag” vista in Figura 7 quando $\lambda_{\min} \ll \lambda_{\max}$.

Prima di procedere diamo una definizione che è molto importante per la comprensione dell'intero metodo

Definizione 4.3. Consideriamo una matrice $A \in \mathbb{R}^{n \times n}$ simmetrica e definita positiva e il funzionale ϕ di Equazione (4.19). Un vettore $\mathbf{x}^{(k)} \in \mathbb{R}^n$ è ottimale rispetto alla direzione $\mathbf{d} \in \mathbb{R}^n$ se

$$(4.22) \quad \phi(\mathbf{x}^{(k)}) \leq \phi(\mathbf{x}^{(k)} + \lambda \mathbf{d}), \quad \forall \lambda \in \mathbb{R}.$$

Per capire meglio l'importanza di questa definizione facciamo la seguente osservazione. Il vettore $\mathbf{d} \in \mathbb{R}^n$ modifica il vettore di partenza $\mathbf{x}^{(k)}$ tramite tutte le sue entrate e il coefficiente λ , i.e.,

$$\mathbf{x}^{(k)} + \lambda \mathbf{d} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} + \lambda \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{pmatrix} = \begin{pmatrix} x_1 + \lambda d_1 \\ x_2 + \lambda d_2 \\ \vdots \\ x_n + \lambda d_n \end{pmatrix}$$

Dire che il vettore $\mathbf{x}^{(k)}$ è ottimale rispetto alla direzione $\mathbf{d}$ comporta che qualsiasi modifica fatta proporzionale a $\mathbf{d}$ non permette di avvicinarsi al minimo di ϕ . Infatti, nella definizione precedente, Equazione (4.22), il valore $\phi(\mathbf{x}^{(k)})$ è minore o uguale al valore che ϕ assume se modificassimo $\mathbf{x}^{(k)}$ nella direzione $\mathbf{d}$.

Idealmente una volta che troviamo uno stato ottimale rispetto a una direzione $\mathbf{d}$ non vorremmo più modificare $\mathbf{x}^{(k)}$ lungo $\mathbf{d}$. Questa è l'idea su cui si basa la procedura iterativa del gradiente coniugato: una volta trovato un vettore ottimale $\mathbf{x}^{(k)}$ rispetto a una direzione $\mathbf{d}$ non lo si deve più modificare lungo questa direzione.

Nel metodo del gradiente questo non è verificato per tutte le iterazioni. Infatti, consideriamo ancora il caso di una matrice $A \in \mathbb{R}^{2 \times 2}$, l'effetto della “convergenza a zig-zag” è dovuta al

fatto che a un certo punto otteniamo un vettore ottimale rispetto a una direzione e poi viene modificato di nuovo lungo la direzione $\mathbf{d}$.

Questo concetto è rappresentato graficamente da Figura 10. Partendo dal vettore $\mathbf{x}^{(1)}$, ottimale rispetto alla direzione $\mathbf{d}_1$, viene creato il vettore $\mathbf{x}^{(2)}$ lungo la direzione $\mathbf{d}_2$. Partendo da $\mathbf{x}^{(2)}$ viene creato il vettore $\mathbf{x}^{(3)}$ partendo dalla direzione $\mathbf{d}_3$ simile rispetto a $\mathbf{d}_1$. Ragionamenti analogo può essere fatto per le iterate successive.

Intuitivamente si può pensare che nel metodo del gradiente coniugato una volta fatta la modifica lungo una direzione, non si vuole più modificare il vettore lungo la stessa direzione. Nell'esempio di prima questo non è così infatti le direzioni $\mathbf{d}_1$ e $\mathbf{d}_3$ sono parallele.

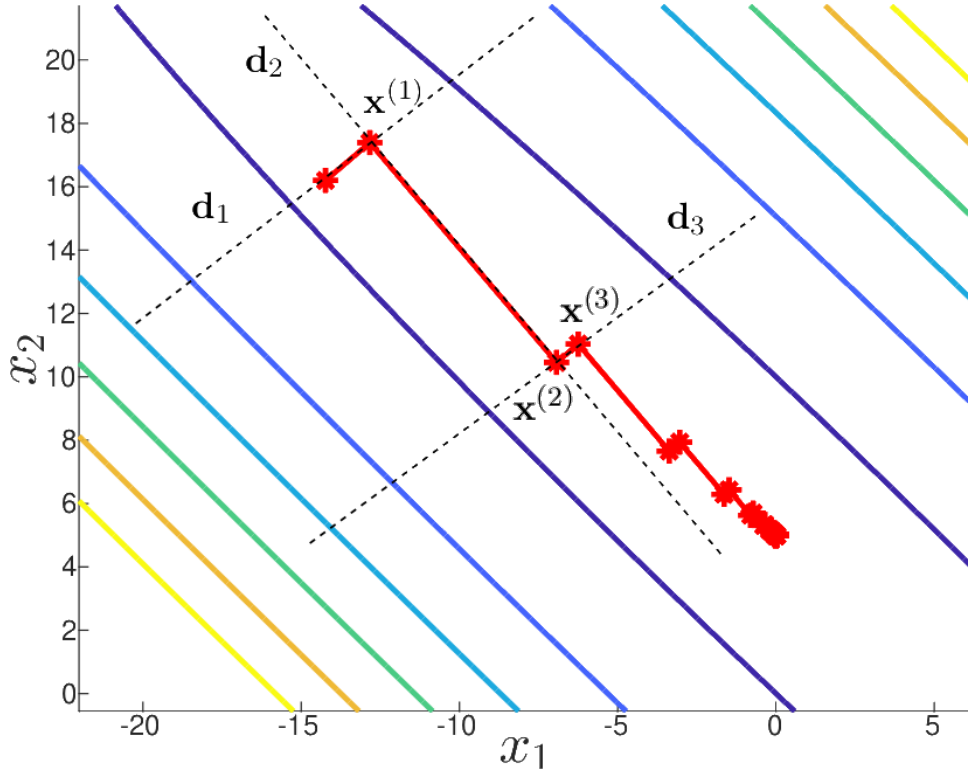


Figura 10. Rappresentazione grafica delle iterazioni del metodo del gradiente. Le direzioni di modifica dei vettori $\mathbf{x}^{(1)}$, $\mathbf{x}^{(2)}$ e $\mathbf{x}^{(3)}$ sono messe in evidenza da linee tratteggiate.

Ci si pone ora il problema di trovare il modo di costruire questa sequenza di vettori $\mathbf{x}^{(k)}$ e fare in modo che non vengano più modificati lungo la direzione per cui sono ottimali.

Cerchiamo ora una condizione più applicativa in modo da identificare e costruire direzioni ottimali.

Proposizione 4.2. *Un vettore $\mathbf{x}^{(k)}$ è ottimale rispetto a una direzione $\mathbf{d} \in \mathbb{R}^n$ se*

$$(4.23) \quad \mathbf{d} \cdot \mathbf{r}^{(k)} = 0.$$

Dimostrazione. Calcoliamo la derivata di ϕ rispetto a λ

$$(4.24) \quad \frac{\partial \phi}{\partial \lambda} (\mathbf{x}^{(k)} + \lambda \mathbf{d}) = \mathbf{d}^t (A\mathbf{x}^{(k)} - \mathbf{b}) + \lambda \mathbf{d}^t A \mathbf{d}.$$

Dato che $\mathbf{x}^{(k)}$ è ottimale rispetto a una direzione $\mathbf{d} \in \mathbb{R}^n$, sappiamo che vale Equazione (4.22). Quindi la funzione ϕ ammette un minimo per $\lambda = 0$ e di conseguenza sappiamo la sua derivata,

Equazione (4.24), valutata in $\lambda = 0$ è zero, i.e.,

$$\mathbf{d}^t (A\mathbf{x}^{(k)} - \mathbf{b}) = 0 \quad \Rightarrow \quad \mathbf{d}^t \mathbf{r}^{(k)} = 0.$$

□

Proposizione 4.3. *Nel metodo del gradiente il vettore $\mathbf{x}^{(k+1)}$ è ottimale rispetto alla direzione $\mathbf{r}^{(k+1)}$, ma $\mathbf{x}^{(k+2)}$ non è più ottimale rispetto alla direzione $\mathbf{r}^{(k)}$.*

Questa proposizione è molto importante. Essa dice che i vettori la posizione $\mathbf{x}^{(k+1)}$ è ottimale rispetto alle due direzioni precedenti, ma la posizione $\mathbf{x}^{(k+2)}$ perde l'ottimalità rispetto alla direzione $\mathbf{r}^{(k)}$. Di conseguenza il metodo del gradiente dovrà sistemare ancora le successive posizioni $\mathbf{x}^{(k+3)}$, $\mathbf{x}^{(k+4)}$, ... in modo che tornino ottimali rispetto tutte alle direzioni precedenti. Da questa proposizione deriva l'andamento a zig-zag verso la soluzione, infatti si deve correggere di volta in volta l'ottimalità che si è persa lungo le varie direzioni.

Diamo ora la procedura per creare questa successione di posizioni ottimali e direzione. Inizializziamo il residuo e la prima direzione, i.e.,

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)} \quad \text{e} \quad \mathbf{d}^{(0)} = \mathbf{r}^{(0)}.$$

Il nuovo punto $\mathbf{x}^{(k+1)}$ sarà dato da

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)},$$

dove

$$\alpha_k := \frac{(\mathbf{d}^{(k)})^t \mathbf{r}^{(k)}}{(\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)}}.$$

Una volta definito la nuova posizione si calcola la direzione con cui verrà costruita la posizione successiva in modo tale che le posizioni che verranno costruite rimangano ottimali con le direzioni precedenti. In particolare abbiamo

$$\mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)} - \beta_k \mathbf{d}^{(k)},$$

dove

$$\beta_k := \frac{(\mathbf{d}^{(k)})^t A \mathbf{r}^{(k+1)}}{(\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)}}.$$

Convergenza. Come abbiamo detto all'inizio di questa sezione il metodo del gradiente coniugato cura la convergenza del metodo del gradiente preservando l'ottimalità dei vettori della successione $\mathbf{x}^{(k)}$ rispetto alle direzioni precedenti.

Questo fatto porta a un notevole vantaggio in termini di iterazioni necessarie alla convergenza. Infatti vale il seguente risultato.

Teorema 4.7. *Sia $A \in \mathbb{R}^{n \times n}$ una matrice simmetrica e definita positiva, il metodo del gradiente coniugato converge in al più n iterazioni.*

Questo risultato è molto importante. Per la prima volta viene fornito un limite superiore alle iterate necessarie per convergere alla soluzione esatta. Infatti nel metodo di Jacobi, Gauß-Seidel, Richardson e perfino nel metodo del gradiente stesso dobbiamo mettere un numero grande arbitrario di iterazioni che non può essere definito a priori dalla dimensione della matrice.

In Figura 11 riportiamo il grafico delle iterazioni necessarie per convergere alla soluzione esatta quando consideriamo un sistema lineare di dimensione 2. Si può vedere come nel metodo del gradiente coniugato la convergenza a zig-zag sparisce.

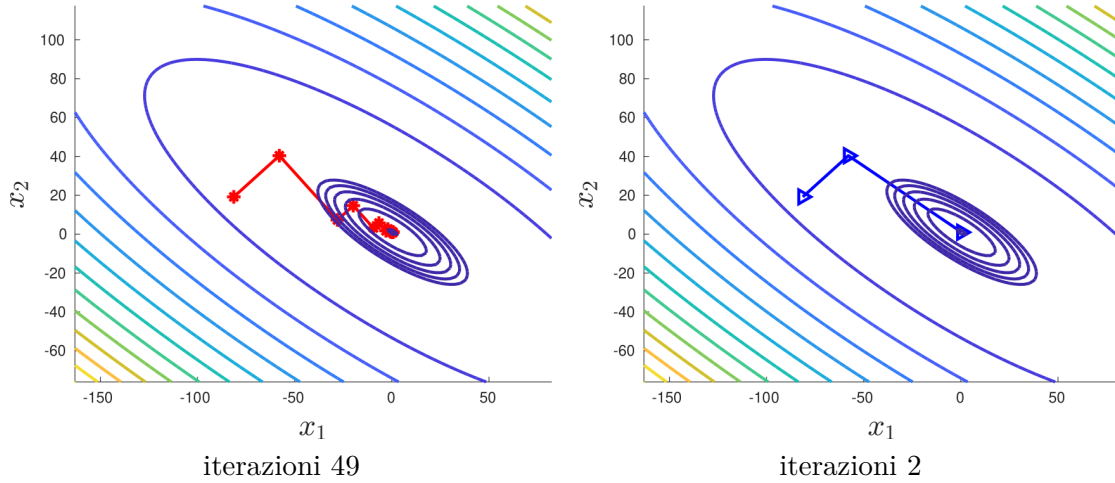


Figura 11. Confronto fra le iterazioni del metodo del gradiente, sinistra, e il metodo del gradiente coniugato, destra.

Rapidità. In questo metodo le operazioni richieste si riducono ancora una volta a operazioni base di matrici: somma di vettori $O(n)$, prodotti di un vettore per uno scalare $O(n)$, prodotti scalari fra vettori $O(n)$ e prodotto matrice-vettori $O(n^2)$.

In una generica iterata dobbiamo calcolare sia il vettore soluzione $\mathbf{x}^{(k+1)}$ sia direzione ottimale per il passaggio successivo $\mathbf{d}^{(k+1)}$. Riassumendo dobbiamo eseguire le seguenti operazioni:

- (1) calcolare il coefficiente

$$\alpha_k := \frac{(\mathbf{d}^{(k)})^t \mathbf{r}^{(k)}}{(\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)}},$$

esso richiede un prodotto scalare per trovare il numeratore $O(n)$, un prodotto matrice vettore per calcolare $\mathbf{y}^{(k)} = A \mathbf{d}^{(k)}$, $O(n^2)$, un altro prodotto scalare per il denominatore $(\mathbf{d}^{(k)})^t \mathbf{y}^{(k)}$, $O(n)$, e una divisione. Quindi abbiamo

$$2O(n) + O(n^2) + 1 = O(n^2).$$

- (2) calcolo della prossima iterata

$$(4.25) \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)},$$

che richiede il prodotto per uno scalare per ogni componente di $\mathbf{d}^{(k)}$, $O(n)$, e una somma di vettori, $O(n)$. Di conseguenza abbiamo un costo totale di $O(n)$.

- (3) abbiamo già visto che il calcolo del residuo richiede solo operazioni elementari fra matrici e uno sforzo in termini di operazioni macchina pari a $O(n^2)$.

- (4) calcolare il coefficiente

$$\beta_k = \frac{(\mathbf{d}^{(k)})^t A \mathbf{r}^{(k+1)}}{(\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)}},$$

richiede il calcolo dei coefficienti

$$(\mathbf{d}^{(k)})^t A \mathbf{r}^{(k+1)} \quad \text{e} \quad (\mathbf{d}^{(k)})^t A \mathbf{d}^{(k)},$$

che sono operazioni simili al denominatore di α_k , quindi richiedono un $O(n^2)$ operazioni, ossia

$$\underbrace{O(n^2)}_{\text{prodotto matrice-vettore}} + \underbrace{O(n)}_{\text{prodotto scalare fra vettori}} = O(n^2).$$

(5) la nuova direzione per il calcolo dell'iterata successiva è data da

$$(4.26) \quad \mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)} - \beta_k \mathbf{d}^{(k)},$$

che ha una struttura simile a quella di Equazione (4.25), quindi abbiamo un costo di $O(n)$.

Sommando tutti questi contributi abbiamo che

$$\underbrace{O(n^2)}_{(1)} + \underbrace{O(n)}_{(2)} + \underbrace{O(n^2)}_{(3)} + \underbrace{O(n^2)}_{(4)} + \underbrace{O(n)}_{(5)} = O(n^2).$$

Nota Implementativa 4.9. In Equazioni (4.25) e (4.26) vengono fatte operazioni simili, quindi dato che sono operazioni uguali e ripetute a ogni iterazione si possono ottimizzare, si veda anche la Nota Implementativa 3.3.

Nota Implementativa 4.10. Dato che non viene di fatto modificata mai la matrice A di partenza si può sfruttare la sua natura sparsa per rendere più veloce il prodotto matrice-vettore $O(n^2)$.

Pseudocodice. Come abbiamo fatto per gli altri processi iterativi vediamo lo pseudocodice per la costruzione della generica iterata del metodo del gradiente.

In Algoritmo 9 mettiamo lo pseudocodice. Questa volta per costruire le iterate abbiamo bisogno di molti più input rispetto a prima: la matrice $A \in \mathbb{R}^{n \times n}$, i vettori $\mathbf{x}^{(k)}$, $\mathbf{r}^{(k)}$ e $\mathbf{d}^{(k)} \in \mathbb{R}^n$. Inoltre notiamo che in output abbiamo: la nuova iterata, $\mathbf{x}^{(k+1)}$, il nuovo residuo, $\mathbf{r}^{(k+1)}$, e la direzione lungo cui calcolare l'iterata successiva, $\mathbf{d}^{(k+1)}$.

Algorithm 9 Procedura per il metodo del gradiente coniugato.

Input: $A \in \mathbb{R}^{n \times n}$, $\mathbf{x}^{(k)}$, $\mathbf{r}^{(k)}$, $\mathbf{d}^{(k)} \in \mathbb{R}^n$;
Output: $\mathbf{x}^{(k+1)}$, $\mathbf{r}^{(k+1)}$, $\mathbf{d}^{(k+1)} \in \mathbb{R}^n$;

- 1: $\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$;
 - 2: $\mathbf{y}^{(k)} = A\mathbf{d}^{(k)}$;
 - 3: $\mathbf{z}^{(k)} = A\mathbf{r}^{(k)}$;
 - 4: $\alpha_k = (\mathbf{d}^{(k)} \cdot \mathbf{r}^{(k)}) / (\mathbf{d}^{(k)} \cdot \mathbf{y}^{(k)})$;
 - 5: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{d}^{(k)}$;
 - 6: $\mathbf{r}^{(k+1)} = \mathbf{b} - A\mathbf{x}^{(k+1)}$;
 - 7: $\mathbf{w}^{(k)} = A\mathbf{r}^{(k+1)}$;
 - 8: $\beta_k = (\mathbf{d}^{(k)} \cdot \mathbf{w}^{(k)}) / (\mathbf{d}^{(k)} \cdot \mathbf{y}^{(k)})$;
 - 9: $\mathbf{d}^{(k+1)} = \mathbf{r}^{(k+1)} - \beta_k \mathbf{d}^{(k)}$;
-

In questo algoritmo possiamo distinguere due parti: da linea 1 a linea 5 dove si determina l'iterata successiva, da linea 6 a linea 9 dove si calcola la direzione per calcolare il prossimo passo.

Nota Implementativa 4.11. *Come abbiamo sottolineato più volte anche in questo caso si utilizzano operazioni base. Quindi se si utilizzano una libreria per le strutture di matrici e vettori conviene utilizzare le operazioni di somma e prodotto implementate all'interno. Oltre a rendere più veloce il codice esse lo rendono anche più leggibile.*

Nota Implementativa 4.12. *Gli input per calcolare l'iterata successiva variano da metodo a metodo. Per salvare il codice e renderlo più flessibile si può pensare di creare una struttura o classe che racchiude tutti gli input. Gli algoritmi descritti avranno in ingresso questa struttura dati che è del tutto generica e compatta. In questo modo il codice diventa più pulito e aggiungere altri input dettati da altri metodi diventa più semplice.*